

第 5 章

几何变换

几何变换是指将一幅图像映射到另外一幅图像内的操作。OpenCV 提供了多个与映射有关的函数，这些函数使用起来方便灵活，能够高效地完成图像的映射。

根据 OpenCV 函数的不同，本章将映射关系划分为缩放、翻转、仿射变换、透视、重映射等。

5.1 缩放

在 OpenCV 中，使用函数 `cv2.resize()` 实现对图像的缩放，该函数的具体形式为：

```
dst = cv2.resize( src, dsize[, fx[, fy[, interpolation]]] )
```

式中：

- `dst` 代表输出的目标图像，该图像的类型与 `src` 相同，其大小为 `dsize`（当该值非零时），或者可以通过 `src.size()`、`fx`、`fy` 计算得到。
- `src` 代表需要缩放的原始图像。
- `dsize` 代表输出图像大小。
- `fx` 代表水平方向的缩放比例。
- `fy` 代表垂直方向的缩放比例。
- `interpolation` 代表插值方式，具体如表 5-1 所示。

表 5-1 插值方式

类型	说明
<code>cv2.INTER_NEAREST</code>	最临近插值
<code>cv2.INTER_LINEAR</code>	双线性插值（默认方式）
<code>cv2.INTER_CUBIC</code>	三次样条插值。首先对源图像附近的 4×4 近邻区域进行三次样条拟合，然后将目标像素对应的三次样条值作为目标图像对应像素点的值
<code>cv2.INTER_AREA</code>	区域插值，根据当前像素点周边区域的像素实现当前像素点的采样。该方法类似最临近插值方式
<code>cv2.INTER_LANCZOS4</code>	一种使用 8×8 近邻的 Lanczos 插值方法
<code>cv2.INTER_LINEAR_EXACT</code>	位精确双线性插值
<code>cv2.INTER_MAX</code>	差值编码掩码
<code>cv2.WARP_FILL_OUTLIERS</code>	标志，填补目标图像中的所有像素。如果它们中的一些对应源图像中的奇异点（离群值），则将它们设置为零



续表

类型	说明
cv2.WARP_INVERSE_MAP	标志，逆变换。 例如，极坐标变换： - 如果 flag 未被设置，则进行转换：$\text{dst}(\emptyset, \rho) = \text{src}(x, y)$ - 如果 flag 被设置，则进行转换：$\text{dst}(x, y) = \text{src}(\emptyset, \rho)$

在 `cv2.resize()` 函数中，目标图像的大小可以通过“参数 `dsizesize`”或者“参数 `fx` 和 `fy`”二者之一来指定，具体介绍如下。

- 情况 1：通过参数 `dsizesize` 指定

如果指定参数 `dsizesize` 的值，则无论是否指定了参数 `fx` 和 `fy` 的值，都由参数 `dsizesize` 来决定目标图像的大小。

此时需要注意的是，`dsizesize` 内第 1 个参数对应缩放后图像的宽度（`width`，即列数 `cols`，与参数 `fx` 相关），第 2 个参数对应缩放后图像的高度（`height`，即行数 `rows`，与参数 `fy` 相关）。

指定参数 `dsizesize` 的值时，`x` 方向的缩放大小（参数 `fx`）为：

```
(double)dsizesize.width/src.cols
```

同时，`y` 方向的缩放大小（参数 `fy`）为：

```
(double)dsizesize.height/src.rows
```

- 情况 2：通过参数 `fx` 和 `fy` 指定

如果参数 `dsizesize` 的值是 `None`，那么目标图像的大小通过参数 `fx` 和 `fy` 来决定。此时，目标图像的大小为：

```
dsizesize=Size(round(fx*src.cols),round(fy*src.rows))
```

插值是指在对图像进行几何处理时，给无法直接通过映射得到值的像素点赋值。例如，将图像放大为原来的 2 倍，必然会多出一些无法被直接映射值的像素点，对于这些像素点，插值方式决定了如何确定它们的值。除此以外，还会存在一些非整数的映射值，例如，反向映射可能会把目标图像中的像素点值映射到原始图像中的非整数位置对应的位置上，当然原始图像内是不可能存在这样的非整数位置的，即目标图像上的该像素点不能对应到原始图像的某个具体位置上，此时也要对这些像素点进行插值处理，以完成映射。

函数 `cv2.resize()` 能实现对原始图像的缩放功能，需要注意的是，开始运算前，操作前的目标图像 `dst` 自身的大小、类型与最终得到的目标图像 `dst` 是没有任何关系的。目标图像 `dst` 的最终大小和类型是通过 `src`、`dsizesize`、`fx`、`fy` 指定的。如果能让原始图像调整为和目标图像一样大，则必须通过上述属性指定。

当缩小图像时，使用区域插值方式（`INTER_AREA`）能够得到最好的效果；当放大图像时，使用三次样条插值（`INTER_CUBIC`）方式和双线性插值（`INTER_LINEAR`）方式都能够取得较好的效果。三次样条插值方式速度较慢，双线性插值方式速度相对较快且效果并不逊色。

【例 5.1】 设计程序，使用函数 `cv2.resize()` 对一个数组进行简单缩放。

为了方便观察，这里我们尝试直接通过函数 `cv2.resize()` 来生成一个与原始数组等大小的数组。

根据题目要求，设计程序如下：

```
import cv2
import numpy as np
img=np.ones([2,4,3],dtype=np.uint8)
size=img.shape[:2]
rst=cv2.resize(img,size)
print("img.shape=\n",img.shape)
print("img=\n",img)
print("rst.shape=\n",rst.shape)
print("rst=\n",rst)
```

在本例中，我们期望通过函数 `cv2.resize()` 对原始图像进行缩放。为了方便观察，将目标图像设置为与原始图像等大小。

运行程序，结果如下：

```
img.shape=
(2, 4, 3)
img=
[[[1 1 1]
  [1 1 1]
  [1 1 1]
  [1 1 1]]

 [[1 1 1]
  [1 1 1]
  [1 1 1]
  [1 1 1]]]
rst.shape=
(4, 2, 3)
rst=
[[[1 1 1]
  [1 1 1]]

 [[1 1 1]
  [1 1 1]]

 [[1 1 1]
  [1 1 1]]

 [[1 1 1]
  [1 1 1]]]
```

通过程序我们观察到，我们的目的没有达到，目标图像的大小与原始图像的大小并不一致。原始图像的大小是 2 行 4 列，目标图像的大小是 4 行 2 列：

- 目标图像的行数是原始图像的列数。
- 目标图像的列数是原始图像的行数。

通过以上例题我们进一步确认：函数 `cv2.resize()` 内 `dsize` 参数与图像 `shape` 属性在行、列的顺序上是不一致的，或者说，

- 在 `shape` 属性中，第 1 个值对应的是行数，第 2 个值对应的是列数。
- 在 `dsize` 参数中，第 1 个值对应的是列数，第 2 个值对应的是行数。

我们通常使用等大小的图像进行测试，在这种情况下，可能无法发现 `cv2.resize()` 函数内 `dsize` 参数的具体使用方式。

在使用 `cv2.resize()` 函数时，要额外注意参数 `dsize` 的属性顺序问题。

【例 5.2】 设计程序，使用函数 `cv2.resize()` 完成一个简单的图像缩放。

根据题目要求，设计程序如下：

```
import cv2
img=cv2.imread("test.bmp")
rows,cols=img.shape[:2]
size=(int(cols*0.9),int(rows*0.5))
rst=cv2.resize(img,size)
print("img.shape=",img.shape)
print("rst.shape=",rst.shape)
```

运行程序，结果如下：

```
img.shape= (512, 51, 3)
rst.shape= (256, 45, 3)
```

从程序可以看出：

- 列数变为原来的 0.9 倍，计算得到 $51 \times 0.9 = 45.9$ ，取整得到 45。
- 行数变为原来的 0.5 倍，计算得到 $512 \times 0.5 = 256$ 。

【例 5.3】 设计程序，控制函数 `cv2.resize()` 的 `fx` 参数、`fy` 参数，完成图像缩放。

根据题目要求，设计程序如下：

```
import cv2
img=cv2.imread("test.bmp")
rst=cv2.resize(img,None,fx=2,fy=0.5)
print("img.shape=",img.shape)
print("rst.shape=",rst.shape)
```

运行程序，结果如下：

```
img.shape= (512, 51, 3)
rst.shape= (256, 102, 3)
```

从程序可以看出：

- `fx` 进行的是水平方向的缩放，将列数变为原来的 2 倍，得到 $51 \times 2 = 102$ 。
- `fy` 进行的是垂直方向的缩放，将行数变为原来的 0.5 倍，得到 $512 \times 0.5 = 256$ 。



5.2 翻转

在 OpenCV 中, 图像的翻转采用函数 `cv2.flip()` 实现, 该函数能够实现图像在水平方向翻转、垂直方向翻转、两个方向同时翻转, 其语法结构为:

```
dst = cv2.flip( src, flipCode )
```

式中:

- `dst` 代表和原始图像具有同样大小、类型的目标图像。
- `src` 代表要处理的原始图像。
- `flipCode` 代表旋转类型。该参数的意义如表 5-2 所示。

表 5-2 `flipCode` 参数的意义

参数值	说明	意义
0	只能是 0	绕着 x 轴翻转
正数	1、2、3 等任意正数	绕着 y 轴翻转
负数	-1、-2、-3 等任意负数	围绕 x 轴、y 轴同时翻转

该函数中, 目标像素点与原始像素点的关系可表述为:

$$dst_{ij} = \begin{cases} src_{src.rows-i-1,j}, & flipCode = 0 \\ src_{i,src.cols-j-1}, & flipCode > 0 \\ src_{src.rows-i-1,src.cols-j-1}, & flipCode < 0 \end{cases}$$

其中, `dst` 是目标像素点, `src` 是原始像素点。

【例 5.4】 设计程序, 使用函数 `cv2.flip()` 完成图像的翻转。

根据题目要求, 设计程序如下:

```
import cv2
img=cv2.imread("lena.bmp")
x=cv2.flip(img,0)
y=cv2.flip(img,1)
xy=cv2.flip(img,-1)
cv2.imshow("img",img)
cv2.imshow("x",x)
cv2.imshow("y",y)
cv2.imshow("xy",xy)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序, 出现如图 5-1 所示的运行结果, 其中:

- 图(a)是原始图像 `lena`。
- 图(b)是语句 `x=cv2.flip(img,0)` 生成的图像, 该图像由图像 `lena` 围绕 x 轴翻转得到。
- 图(c)是语句 `y=cv2.flip(img,1)` 生成的图像, 该图像由图像 `lena` 围绕 y 轴翻转得到。
- 图(d)是语句 `xy=cv2.flip(img,-1)` 生成的图像, 该图像由图像 `lena` 围绕 x 轴、y 轴翻转得到。



图 5-1 【例 5.4】程序的运行结果

5.3 仿射

仿射变换是指图像可以通过一系列的几何变换来实现平移、旋转等多种操作。该变换能够保持图像的平直性和平行性。平直性是指图像经过仿射变换后，直线仍然是直线；平行性是指图像在完成仿射变换后，平行线仍然是平行线。

OpenCV 中的仿射函数为 `cv2.warpAffine()`，其通过一个变换矩阵(映射矩阵) M 实现变换，具体为：

$$\text{dst}(x, y) = \text{src}(M_{11}x + M_{12}y + M_{13}, M_{21}x + M_{22}y + M_{23})$$

如图 5-2 所示，可以通过一个变换矩阵 M ，将原始图像 O 变换为仿射图像 R 。

仿射图像 $R = \text{变换矩阵 } M \times \text{原始图像 } O$

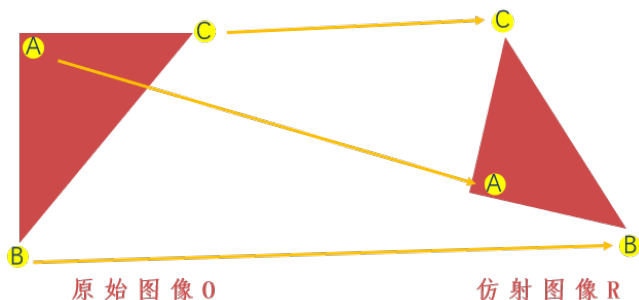


图 5-2 仿射变换

因此，可以采用仿射函数 `cv2.warpAffine()`实现对图像的旋转，该函数的语法格式如下：

```
dst = cv2.warpAffine( src, M, dsize[, flags[, borderMode[, borderValue]]] )
```

式中：

- `dst` 代表仿射后的输出图像，该图像的类型和原始图像的类型相同。`dsize` 决定输出图像的实际大小。
- `src` 代表要仿射的原始图像。
- `M` 代表一个 2×3 的变换矩阵。使用不同的变换矩阵，就可以实现不同的仿射变换。
- `dsize` 代表输出图像的尺寸大小。

- `flags` 代表插值方法，默认为 `INTER_LINEAR`。当该值为 `WARP_INVERSE_MAP` 时，意味着 M 是逆变换类型，实现从目标图像 `dst` 到原始图像 `src` 的逆变换。具体可选值参见表 5-1。
- `borderMode` 代表边类型，默认为 `BORDER_CONSTANT`。当该值为 `BORDER_TRANSPARENT` 时，意味着目标图像内的值不做改变，这些值对应原始图像内的异常值。
- `borderValue` 代表边界值，默认是 0。

通过以上分析可知，在 OpenCV 中使用函数 `cv2.warpAffine()` 实现仿射变换，忽略其可选参数后的语法格式为：

```
dst = cv2.warpAffine( src , M , dsize )
```

其通过转换矩阵 M 将原始图像 `src` 转换为目标图像 `dst`：

$$\text{dst}(x, y) = \text{src}(M_{11}x + M_{12}y + M_{13}, M_{21}x + M_{22}y + M_{23})$$

因此，进行何种形式的仿射变换完全取决于转换矩阵 M 。下面分别介绍通过不同的转换矩阵 M 实现的不同的仿射变换。

5.3.1 平移

通过转换矩阵 M 实现将原始图像 `src` 转换为目标图像 `dst`：

$$\text{dst}(x, y) = \text{src}(M_{11}x + M_{12}y + M_{13}, M_{21}x + M_{22}y + M_{23})$$

将原始图像 `src` 向右侧移动 100 个像素、向下方移动 200 个像素，则其对应关系为：

$$\text{dst}(x, y) = \text{src}(x + 100, y + 200)$$

将上述表达式补充完整，即：

$$\text{dst}(x, y) = \text{src}(1 \cdot x + 0 \cdot y + 100, 0 \cdot x + 1 \cdot y + 200)$$

根据上述表达式，可以确定对应的转换矩阵 M 中各个元素的值为：

- $M_{11}=1$
- $M_{12}=0$
- $M_{13}=100$
- $M_{21}=0$
- $M_{22}=1$
- $M_{23}=200$

将上述值代入转换矩阵 M ，得到：

$$M = \begin{bmatrix} 1 & 0 & 100 \\ 0 & 1 & 200 \end{bmatrix}$$

在已知转换矩阵 M 的情况下，可以直接利用转换矩阵 M 调用函数 `cv2.warpAffine()` 完成图像的平移。

【例 5.5】 设计程序，利用自定义转换矩阵完成图像平移。

根据题目要求，设计程序如下：

```
import cv2
import numpy as np
img=cv2.imread("lena.bmp")
height,width=img.shape[:2]
x=100
y=200
M = np.float32([[1, 0, x], [0, 1, y]])
move=cv2.warpAffine(img,M,(width,height))
cv2.imshow("original",img)
cv2.imshow("move",move)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，出现如图 5-3 所示的运行结果，其中左图是原始图像，右图是移动结果图像。

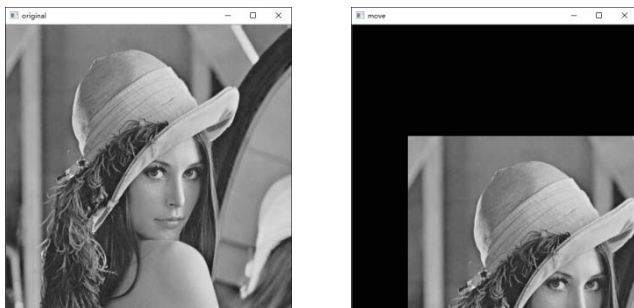


图 5-3 【例 5.5】程序的运行结果

5.3.2 旋转

在使用函数 `cv2.warpAffine()` 对图像进行旋转时，可以通过函数 `cv2.getRotationMatrix2D()` 获取转换矩阵。该函数的语法格式为：

```
retval=cv2.getRotationMatrix2D(center, angle, scale)
```

式中：

- `center` 为旋转的中心点。
- `angle` 为旋转角度，正数表示逆时针旋转，负数表示顺时针旋转。
- `scale` 为变换尺度（缩放大小）。

利用函数 `cv2.getRotationMatrix2D()` 可以直接生成要使用的转换矩阵 `M`。例如，想要以图像中心为圆点，逆时针旋转 45° ，并将目标图像缩小为原始图像的 0.6 倍，则在调用函数 `cv2.getRotationMatrix2D()` 生成转换矩阵 `M` 时所使用的语句为：

```
M=cv2.getRotationMatrix2D((height/2,width/2),45,0.6)
```

【例 5.6】设计程序，完成图像旋转。

根据题目要求，设计程序如下：

```
import cv2
```




```

img=cv2.imread("lena.bmp")
height,width=img.shape[:2]
M=cv2.getRotationMatrix2D((width/2,height/2),45,0.6)
rotate=cv2.warpAffine(img,M,(width,height))
cv2.imshow("original",img)
cv2.imshow("rotation",rotate)
cv2.waitKey()
cv2.destroyAllWindows()

```

运行程序，出现如图 5-4 所示的运行结果，其中左图是原始图像，右图是旋转结果图像。

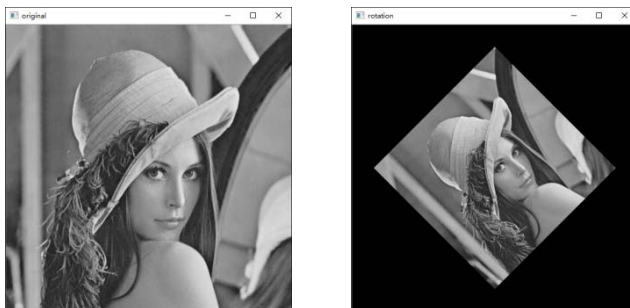


图 5-4 【例 5.6】程序的运行结果

5.3.3 更复杂的仿射变换

5.3.1 节和 5.3.2 节讲的两种仿射变换都比较简单，对于更复杂仿射变换，OpenCV 提供了函数 `cv2.getAffineTransform()` 来生成仿射函数 `cv2.warpAffine()` 所使用的转换矩阵 `M`。该函数的语法格式为：

```
retval=cv2.getAffineTransform(src, dst)
```

式中：

- `src` 代表输入图像的三个点坐标。
- `dst` 代表输出图像的三个点坐标。

在该函数中，其参数值 `src` 和 `dst` 是包含三个二维数组(x, y)点的数组。上述参数通过函数 `cv2.getAffineTransform()` 定义了两个平行四边形。`src` 和 `dst` 中的三个点分别对应平行四边形的左上角、右上角、左下角三个点。函数 `cv2.warpAffine()` 以函数 `cv2.getAffineTransform()` 获取的转换矩阵 `M` 为参数，将 `src` 中的点仿射到 `dst` 中。函数 `cv2.getAffineTransform()` 对所指定的点完成映射后，将所有其他点的映射关系按照指定点的关系计算确定。

【例 5.7】 设计程序，完成图像仿射。

根据题目要求，设计程序如下：

```

import cv2
import numpy as np
img=cv2.imread('lena.bmp')
rows,cols,ch=img.shape
p1=np.float32([[0,0],[cols-1,0],[0,rows-1]])

```

```

p2=np.float32([[0,rows*0.33],[cols*0.85,rows*0.25],[cols*0.15,rows*0.7]])
M=cv2.getAffineTransform(p1,p2)
dst=cv2.warpAffine(img,M,(cols,rows))
cv2.imshow("original",img)
cv2.imshow("result",dst)
cv2.waitKey()
cv2.destroyAllWindows()

```

在本例中，首先构造了两个三分量的点集合 $p1$ 和 $p2$ ，分别用来指代原始图像和目标图像内平行四边形的三个顶点(左上角、右上角、左下角)。然后使用 $M=cv2.getAffineTransform(p1,p2)$ 获取转换矩阵 M 。接下来， $dst=cv2.warpAffine(img,M,(cols,rows))$ 完成了从原始图像到目标图像的仿射。

运行程序，出现如图 5-5 所示的运行结果，其中左图是原始图像，右图是仿射结果图像。

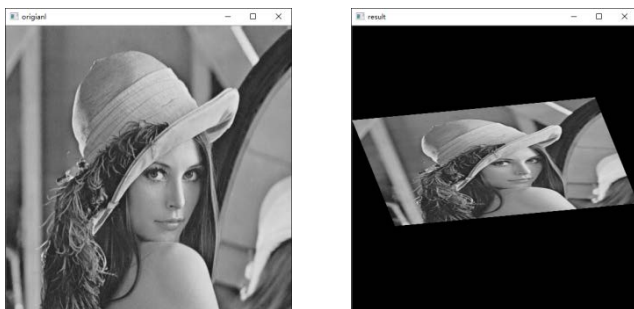


图 5-5 【例 5.7】程序的运行结果

5.4 透视

5.3 节所讲的仿射变换可以将矩形映射为任意平行四边形，透视变换则可以将矩形映射为任意四边形。

透视变换通过函数 `cv2.warpPerspective()` 实现，该函数的语法是：

```
dst = cv2.warpPerspective( src, M, dsize[, flags[, borderMode[, borderValue]]] )
```

式中：

- dst 代表透视处理后的输出图像，该图像和原始图像具有相同的类型。 $dsize$ 决定输出图像的实际大小。
- src 代表要透视的图像。
- M 代表一个 3×3 的变换矩阵。
- $dsize$ 代表输出图像的尺寸大小。
- $flags$ 代表插值方法，默认为 `INTER_LINEAR`。当该值为 `WARP_INVERSE_MAP` 时，意味着 M 是逆变换类型，能实现从目标图像 dst 到原始图像 src 的逆变换。具体可选值参见表 5-1。
- $borderMode$ 代表边类型，默认为 `BORDER_CONSTANT`。当该值为 `BORDER_TRANSPARENT` 时，意味着目标图像内的值不做改变，这些值对应原始图像内的异常值。

- `borderValue` 代表边界值，默认是 0。

与仿射变换一样，同样可以使用一个函数来生成函数 `cv2.warpPerspective()` 所使用的转换矩阵。该函数是 `cv2.getPerspectiveTransform()`，其语法格式为：

```
retval = cv2.getPerspectiveTransform( src, dst )
```

式中：

- `src` 代表输入图像的四个顶点的坐标。
- `dst` 代表输出图像的四个顶点的坐标。

需要注意的是，`src` 参数和 `dst` 参数是包含四个点的数组，与仿射变换函数 `cv2.getAffineTransform()` 中的三个点是不同的。实际使用中，我们可以根据需求控制 `src` 中的四个点映射到 `dst` 中的四个点。

【例 5.8】 设计程序，完成图像透视。

根据题目要求，设计程序如下：

```
import cv2
import numpy as np
img=cv2.imread('demo.bmp')
rows,cols=img.shape[:2]
print(rows,cols)
pts1 = np.float32([[150,50],[400,50],[60,450],[310,450]])
pts2 = np.float32([[50,50],[rows-50,50],[50,cols-50],[rows-50,cols-50]])
M=cv2.getPerspectiveTransform(pts1,pts2)
dst=cv2.warpPerspective(img,M,(cols,rows))
cv2.imshow("img",img)
cv2.imshow("dst",dst)
cv2.waitKey()
cv2.destroyAllWindows()
```

在本例中，指定原始图像中平行四边形的四个顶点 `pts1`，指定目标图像中矩形的四个顶点 `pts2`，使用 `M=cv2.getPerspectiveTransform(pts1,pts2)` 生成转换矩阵 `M`。接下来，使用语句 `dst=cv2.warpPerspective(img,M,(cols,rows))` 完成从平行四边形到矩形的转换。

运行程序，出现如图 5-6 所示的运行结果。其中，左图是原始图像，是一个平行四边形（内含多个小平行四边形）；右图是透视结果图像，是一个矩形（内含多个小矩形）。

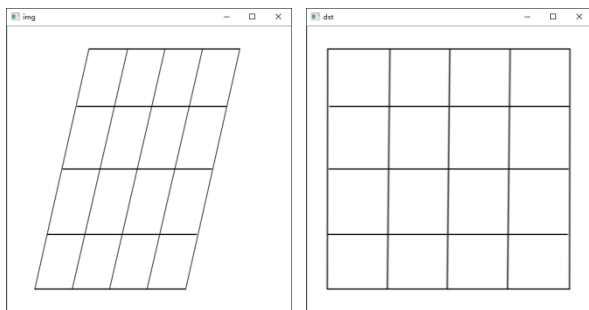


图 5-6 【例 5.8】程序的运行结果

5.5 重映射

把一幅图像内的像素点放置到另外一幅图像内的指定位置, 这个过程称为重映射。OpenCV 提供了多种重映射方式, 但是我们有时会希望使用自定义的方式来完成重映射。

OpenCV 内的重映射函数 `cv2.remap()` 提供了更方便、更自由的映射方式, 其语法格式如下:

```
dst = cv2.remap( src, map1, map2, interpolation[, borderMode[, borderValue]] )
```

式中:

- `dst` 代表目标图像, 它和 `src` 具有相同的大小和类型。
- `src` 代表原始图像。
- `map1` 参数有两种可能的值:
 - 表示(x,y)点的一个映射。
 - 表示 `CV_16SC2`, `CV_32FC1`, `CV_32FC2` 类型(x,y)点的 x 值。
- `map2` 参数同样有两种可能的值:
 - 当 `map1` 表示(x,y)时, 该值为空。
 - 当 `map1` 表示(x,y)点的 x 值时, 该值是 `CV_16UC1`, `CV_32FC1` 类型(x,y)点的 y 值。
- `Interpolation` 代表插值方式, 这里不支持 `INTER_AREA` 方法。具体值参见表 5-1。
- `borderMode` 代表边界模式。当该值为 `BORDER_TRANSPARENT` 时, 表示目标图像内的对应源图像内奇异点 (outliers) 的像素不会被修改。
- `borderValue` 代表边界值, 该值默认为 0。

5.5.1 映射参数的理解

重映射通过修改像素点的位置得到一幅新图像。在构建新图像时, 需要确定新图像中每个像素点在原始图像中的位置。因此, 映射函数的作用是查找新图像像素在原始图像内的位置。该过程是将新图像像素映射到原始图像的过程, 因此被称为反向映射。在函数 `cv2.remap()` 中, 参数 `map1` 和参数 `map2` 用来说明反向映射, `map1` 针对的是坐标 x, `map2` 针对的是坐标 y。

需要说明的是, `map1` 和 `map2` 的值都是浮点数。因此, 目标图像可以映射回一个非整数的值, 这意味着目标图像可以“反向映射”到原始图像中两个像素点之间的位置 (当然, 该位置是不存在像素值的)。这时, 可以采用不同的方法实现插值, 函数中的 `interpolation` 参数可以控制插值方式。正是由于参数 `map1` 和参数 `map2` 的值是浮点数, 所以通过函数 `cv2.remap()` 所能实现的映射关系变得更加随意, 可以通过自定义映射参数实现不同形式的映射。

需要注意的是, 函数 `cv2.remap()` 中参数 `map1` 指代的是像素点所在位置的列号, 参数 `map2` 指代的是像素点所在位置的行号。例如, 我们想将目标图像 (映射结果图像) 中某个点 A 映射为原始图像内处于第 0 行第 3 列上的像素点 B, 那么需要将 A 点所对应的参数 `map1` 对应位置上的值设为 3, 参数 `map2` 对应位置上的值设为 0。所以, 通常情况下, 我们将 `map1` 写为 `mapx`, 并且将 `map2` 写成 `mapy`, 以方便理解。

同样, 如果想将目标图像 (映射结果图像) 中所有像素点都映射为原始图像内处于第 0 行第 3 列上的像素点 B, 那么需要将参数 `map1` 内的值均设为 3, 将参数 `map2` 内的值均设为 0。



【例 5.9】设计程序，使用 `cv2.remap()` 完成数组映射，将目标数组内的所有像素点都映射为原始图像内第 0 行第 3 列上的像素点，以此来了解函数 `cv2.remap()` 内参数 `map1` 和 `map2` 的使用情况。

根据题目要求，可以确定：

- 用来指定列的参数 `map1` (`mapx`) 内的值均为 3。
- 用来指定行的参数 `map2` (`mapy`) 内的值均为 0。

根据题目要求及上述分析，设计程序如下：

```
import cv2
import numpy as np
img=np.random.randint(0,256,size=[4,5],dtype=np.uint8)
rows,cols=img.shape
mapx = np.ones(img.shape,np.float32)*3
mapy = np.ones(img.shape,np.float32)*0
rst=cv2.remap(img,mapx,mapy,cv2.INTER_LINEAR)
print("img=\n",img)
print("mapx=\n",mapx)
print("mapy=\n",mapy)
print("rst=\n",rst)
```

运行程序，出现如下结果：

```
img=
[[120 183 101 252 219]
 [ 51 106 168 221 118]
 [147 16  3 14 159]
 [219 67 254 16 62]]
mapx=
[[3. 3. 3. 3. 3.]
 [3. 3. 3. 3. 3.]
 [3. 3. 3. 3. 3.]
 [3. 3. 3. 3. 3.]]
mapy=
[[0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]]
rst=
[[252 252 252 252 252]
 [252 252 252 252 252]
 [252 252 252 252 252]
 [252 252 252 252 252]]
```

通过观察上述结果可知，目标图像（数组）`dst` 内的所有值都来源于原始图像中第 0 行第 3 列上的像素值 252。

5.5.2 复制

为了更好地了解重映射函数 `cv2.remap()` 的使用方法, 本节介绍如何通过该函数实现图像的复制。在映射时, 将参数进行如下处理:

- 将 `map1` 的值设定为对应位置上的 `x` 轴坐标值。
- 将 `map2` 的值设定为对应位置上的 `y` 轴坐标值。

通过上述处理后, 可以让函数 `cv2.remap()` 实现图像复制。下面通过一个例题来观察实现复制时, 如何设置函数 `cv2.remap()` 内的 `map1` 和 `map2` 参数的值。

【例 5.10】设计程序, 使用函数 `cv2.remap()` 完成数组复制, 了解函数 `cv2.remap()` 内参数 `map1` 和 `map2` 的使用情况。

这里为了方便理解, 将参数 `map1` 定义为 `mapx`, 将参数 `map2` 定义为 `mapy`。后续程序中都采用了这种定义方式, 后面不再重复说明。

根据题目要求, 设计程序如下:

```
import cv2
import numpy as np
img=np.random.randint(0,256,size=[4,5],dtype=np.uint8)
rows,cols=img.shape
mapx = np.zeros(img.shape,np.float32)
mapy = np.zeros(img.shape,np.float32)
for i in range(rows):
    for j in range(cols):
        mapx.itemset((i,j),j)
        mapy.itemset((i,j),i)
rst=cv2.remap(img,mapx,mapy,cv2.INTER_LINEAR)
print("img=\n",img)
print("mapx=\n",mapx)
print("mapy=\n",mapy)
print("rst=\n",rst)
```

运行程序, 出现如下结果:

```
img=
[[203 192 82 156 8]
 [245 157 191 163 2]
 [148 68 46 3 91]
 [227 32 35 123 240]]
mapx=
[[0. 1. 2. 3. 4.]
 [0. 1. 2. 3. 4.]
 [0. 1. 2. 3. 4.]
 [0. 1. 2. 3. 4.]]
mapy=
[[0. 0. 0. 0. 0.]
 [1. 1. 1. 1. 1.]
```



```

[2. 2. 2. 2. 2.]
[3. 3. 3. 3. 3.]]
rst=
[[203 192 82 156 8]
 [245 157 191 163 2]
 [148 68 46 3 91]
 [227 32 35 123 240]]

```

通过本例可以观察到，参数 `mapx` 和参数 `mapy` 分别设置了 `x` 轴方向的坐标和 `y` 轴方向的坐标。函数 `cv2.remap()` 利用参数 `mapx`、`mapy` 所组成的数组构造的映射关系实现了图像的复制。

例如，`rst` 中的像素点 `[3,4]` 在 `src` 内的 `x`、`y` 轴坐标如下：

- `x` 轴坐标取决于 `mapx` 中 `mapx[3,4]` 的值，为 4。
- `y` 轴坐标取决于 `mapy` 中 `mapy[3,4]` 的值，为 3。

这说明 `rst[3,4]` 来源于原始图像 `src` 的第 4 列（`x` 轴方向，由 `mapx[3,4]` 决定）、第 3 行（`y` 轴方向，由 `mapy[3,4]` 决定），即 `rst[3,4]=src[3,4]`。原始对象 `src[3,4]` 的值为 240，所以目标对象 `rst[3,4]` 的值为 240。

【例 5.11】 设计程序，使用函数 `cv2.remap()` 完成图像复制。

根据题目要求，设计程序如下：

```

import cv2
import numpy as np
img=cv2.imread("lena.bmp")
rows,cols=img.shape[:2]
mapx = np.zeros(img.shape[:2],np.float32)
mapy = np.zeros(img.shape[:2],np.float32)
for i in range(rows):
    for j in range(cols):
        mapx.itemset((i,j),j)
        mapy.itemset((i,j),i)
rst=cv2.remap(img,mapx,mapy,cv2.INTER_LINEAR)
cv2.imshow("original",img)
cv2.imshow("result",rst)
cv2.waitKey()
cv2.destroyAllWindows()

```

运行程序，出现如图 5-7 所示的运行结果，其中，左图是原始图像，右图是复制结果图像。

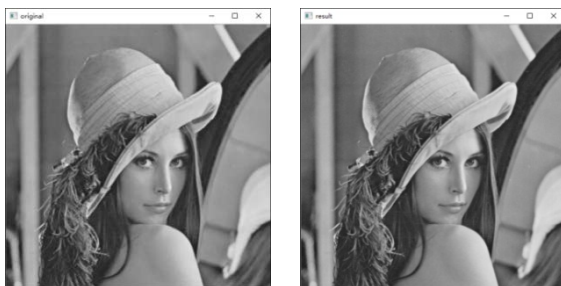


图 5-7 【例 5.11】程序的运行结果

5.5.3 绕x轴翻转

如果想让图像绕着 x 轴翻转，意味着在映射过程中：

- x 坐标轴的值保持不变。
- y 坐标轴的值以 x 轴为对称轴进行交换。

反映在 map1 和 map2 上：

- map1 的值保持不变。
- map2 的值调整为“总行数-1-当前行号”。

需要注意，OpenCV 中行号的下标是从 0 开始的，所以在对称关系中存在“当前行号+对称行号=总行数-1”的关系。据此，在绕着 x 轴翻转时，map2 中当前行的行号调整为“总行数-1-当前行号”。

【例 5.12】设计程序，使用函数 cv2.remap()实现数组绕 x 轴翻转。

根据题目要求，设计程序如下：

```
import cv2
import numpy as np
img=np.random.randint(0,256,size=[4,5],dtype=np.uint8)
rows,cols=img.shape
mapx = np.zeros(img.shape,np.float32)
mapy = np.zeros(img.shape,np.float32)
for i in range(rows):
    for j in range(cols):
        mapx.itemset((i,j),j)
        mapy.itemset((i,j),rows-1-i)
rst=cv2.remap(img,mapx,mapy,cv2.INTER_LINEAR)
print("img=\n",img)
print("mapx=\n",mapx)
print("mapy=\n",mapy)
print("rst=\n",rst)
```

运行程序，出现如下结果：

```
img=
[[119 246 183 18 135]
 [ 58 156 139 182 254]
 [ 45 61 211 214 5]
 [124 208 230 165 224]]
mapx=
[[0. 1. 2. 3. 4.]
 [0. 1. 2. 3. 4.]
 [0. 1. 2. 3. 4.]
 [0. 1. 2. 3. 4.]]
mapy=
[[3. 3. 3. 3. 3.]
 [2. 2. 2. 2. 2.]
```




```
[1. 1. 1. 1. 1.]
[0. 0. 0. 0. 0.]]
rst=
[[124 208 230 165 224]
 [ 45  61 211 214   5]
 [ 58 156 139 182 254]
 [119 246 183  18 135]]
```

【例 5.13】 设计程序，使用函数 `cv2.remap()` 实现图像绕 x 轴的翻转。

根据题目要求，设计程序如下：

```
import cv2
import numpy as np
img=cv2.imread("lena.bmp")
rows,cols=img.shape[:2]
mapx = np.zeros(img.shape[:2],np.float32)
mapy = np.zeros(img.shape[:2],np.float32)
for i in range(rows):
    for j in range(cols):
        mapx.itemset((i,j),j)
        mapy.itemset((i,j),rows-1-i)
rst=cv2.remap(img,mapx,mapy,cv2.INTER_LINEAR)
cv2.imshow("original",img)
cv2.imshow("result",rst)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，出现如图 5-8 所示的运行结果，其中，左图是原始图像，右图是翻转结果图像。

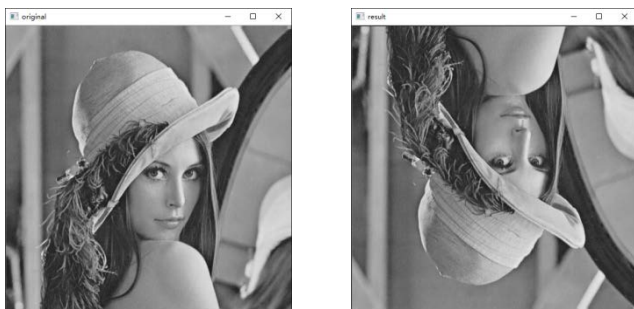


图 5-8 【例 5.13】程序的运行结果

5.5.4 绕y轴翻转

如果想让图像绕着 y 轴翻转，意味着在映射过程中：

- y 坐标轴的值保持不变。
- x 坐标轴的值以 y 轴为对称轴进行交换。

反映在 `map1` 和 `map2` 上：

- `map2` 的值保持不变。

- map1 的值调整为“总列数-1-当前列号”。

需要注意，OpenCV 中列号的下标是从 0 开始的，所以在对称关系中存在“当前列号+对称列号=总列数-1”的关系。据此，在绕着 y 轴翻转时，map1 中当前列的列号调整为“总列数-1-当前列号”。

【例 5.14】设计程序，使用函数 cv2.remap()实现数组绕 y 轴翻转。

根据题目要求，设计程序如下：

```
import cv2
import numpy as np
img=np.random.randint(0,256,size=[4,5],dtype=np.uint8)
rows,cols=img.shape
mapx = np.zeros(img.shape,np.float32)
mapy = np.zeros(img.shape,np.float32)
for i in range(rows):
    for j in range(cols):
        mapx.itemset((i,j),cols-1-j)
        mapy.itemset((i,j),i)
rst=cv2.remap(img,mapx,mapy,cv2.INTER_LINEAR)
print("img=\n",img)
print("mapx=\n",mapx)
print("mapy=\n",mapy)
print("rst=\n",rst)
```

运行程序，出现如下结果：

```
img=
[[210 103 93 51 237]
 [209 17 208 110 116]
 [136 77 4 65 14]
 [244 43 15 135 200]]
mapx=
[[4. 3. 2. 1. 0.]
 [4. 3. 2. 1. 0.]
 [4. 3. 2. 1. 0.]
 [4. 3. 2. 1. 0.]]
mapy=
[[0. 0. 0. 0. 0.]
 [1. 1. 1. 1. 1.]
 [2. 2. 2. 2. 2.]
 [3. 3. 3. 3. 3.]]
rst=
[[237 51 93 103 210]
 [116 110 208 17 209]
 [14 65 4 77 136]
 [200 135 15 43 244]]
```

【例 5.15】设计程序，使用函数 cv2.remap()实现图像绕 y 轴的翻转。



根据题目要求，设计程序如下：

```
import cv2
import numpy as np
img=cv2.imread("lena.bmp")
rows,cols=img.shape[:2]
mapx = np.zeros(img.shape[:2],np.float32)
mapy = np.zeros(img.shape[:2],np.float32)
for i in range(rows):
    for j in range(cols):
        mapx.itemset((i,j),cols-1-j)
        mapy.itemset((i,j),i)
rst=cv2.remap(img,mapx,mapy,cv2.INTER_LINEAR)
cv2.imshow("original",img)
cv2.imshow("result",rst)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，出现如图 5-9 所示的运行结果，其中，左图是原始图像，右图是翻转结果图像。

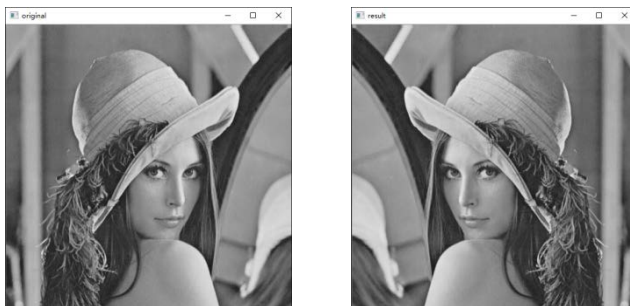


图 5-9 【例 5.15】程序的运行结果

5.5.5 绕x轴、y轴翻转

如果想让图像绕着 x 轴、y 轴翻转，意味着在映射过程中：

- x 坐标轴的值以 y 轴为对称轴进行交换。
- y 坐标轴的值以 x 轴为对称轴进行交换。

反映在 map1 和 map2 上：

- map1 的值调整为“总列数-1-当前列号”。
- map2 的值调整为“总行数-1-当前行号”。

【例 5.16】设计程序，使用函数 cv2.remap()实现数组绕 x 轴、y 轴翻转。

根据题目要求，设计程序如下：

```
import cv2
import numpy as np
img=np.random.randint(0,256,size=[4,5],dtype=np.uint8)
rows,cols=img.shape
```

```

mapx = np.zeros(img.shape,np.float32)
mapy = np.zeros(img.shape,np.float32)
for i in range(rows):
    for j in range(cols):
        mapx.itemset((i,j),cols-1-j)
        mapy.itemset((i,j),rows-1-i)
rst=cv2.remap(img,mapx,mapy,cv2.INTER_LINEAR)
print("img=\n",img)
print("mapx=\n",mapx)
print("mapy=\n",mapy)
print("rst=\n",rst)

```

运行程序，出现如下结果：

```

img=
[[136  69 134 207 132]
 [149 182 170 132  39]
 [183  93 235 252 228]
 [ 34 163  68  69 183]]
mapx=
[[4.  3.  2.  1.  0.]
 [4.  3.  2.  1.  0.]
 [4.  3.  2.  1.  0.]
 [4.  3.  2.  1.  0.]]
mapy=
[[3.  3.  3.  3.  3.]
 [2.  2.  2.  2.  2.]
 [1.  1.  1.  1.  1.]
 [0.  0.  0.  0.  0.]]
rst=
[[183  69  68 163  34]
 [228 252 235  93 183]
 [ 39 132 170 182 149]
 [132 207 134  69 136]]

```

【例 5.17】设计程序，使用函数 `cv2.remap()` 实现图像绕 x 轴、y 轴翻转。

根据题目要求，设计程序如下：

```

import cv2
import numpy as np
img=cv2.imread("lena.bmp")
rows,cols=img.shape[:2]
mapx = np.zeros(img.shape[:2],np.float32)
mapy = np.zeros(img.shape[:2],np.float32)
for i in range(rows):
    for j in range(cols):
        mapx.itemset((i,j),cols-1-j)
        mapy.itemset((i,j),rows-1-i)
rst=cv2.remap(img,mapx,mapy,cv2.INTER_LINEAR)

```



```

cv2.imshow("original",img)
cv2.imshow("result",rst)
cv2.waitKey()
cv2.destroyAllWindows()

```

运行程序，出现如图 5-10 所示的运行结果，其中，左图是原始图像，右图是翻转结果图像。

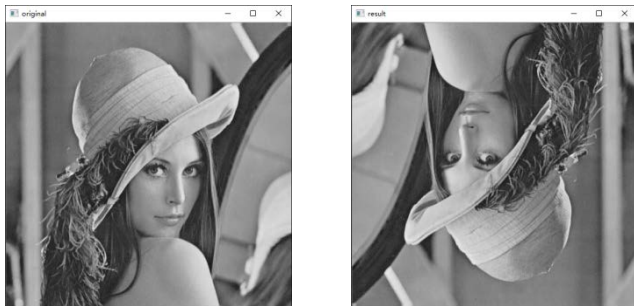


图 5-10 【例 5.17】程序的运行结果

5.5.6 x轴、y轴互换

如果想让图像的 x 轴、y 轴互换，意味着在映射过程中，对于任意一点，都需要将其 x 轴、y 轴坐标互换。

反映在 mapx 和 mapy 上：

- mapx 的值调整为所在行的行号。
- mapy 的值调整为所在列的列号。

需要注意的是，如果行数和列数不一致，上述运算可能存在值无法映射的情况。默认情况下，无法完成映射的值会被处理为 0。

【例 5.18】设计程序，使用函数 cv2.remap() 实现数组的 x 轴、y 轴互换。

根据题目要求，设计程序如下：

```

import cv2
import numpy as np
img=np.random.randint(0,256,size=[4,6],dtype=np.uint8)
rows,cols=img.shape
mapx = np.zeros(img.shape,np.float32)
mapy = np.zeros(img.shape,np.float32)
for i in range(rows):
    for j in range(cols):
        mapx.itemset((i,j),i)
        mapy.itemset((i,j),j)
rst=cv2.remap(img,mapx,mapy,cv2.INTER_LINEAR)
print("img=\n",img)
print("mapx=\n",mapx)
print("mapy=\n",mapy)

```

```
print("rst=\n",rst)
```

运行程序，出现如下结果：

```
img=
[[132 169 35 72 229 221]
 [ 35 19 116 178 12 143]
 [ 72 168 63 36 148 197]
 [ 34 229 3 170 105 64]]
mapx=
[[0. 0. 0. 0. 0. 0.]
 [1. 1. 1. 1. 1. 1.]
 [2. 2. 2. 2. 2. 2.]
 [3. 3. 3. 3. 3. 3.]]
mapy=
[[0. 1. 2. 3. 4. 5.]
 [0. 1. 2. 3. 4. 5.]
 [0. 1. 2. 3. 4. 5.]
 [0. 1. 2. 3. 4. 5.]]
rst=
[[132 35 72 34 0 0]
 [169 19 168 229 0 0]
 [ 35 116 63 3 0 0]
 [ 72 178 36 170 0 0]]
```

在本例中，行数和列数不一致，运算中存在值无法映射的情况。通过程序运行结果可知，无法完成映射的值都被处理为0了。

【例 5.19】设计程序，使用函数 `cv2.remap()` 实现图像的 x 轴、y 轴互换。

根据题目要求，设计程序如下：

```
import cv2
import numpy as np
img=cv2.imread("lena.bmp")
rows,cols=img.shape[:2]
mapx = np.zeros(img.shape[:2],np.float32)
mapy = np.zeros(img.shape[:2],np.float32)
for i in range(rows):
    for j in range(cols):
        mapx.itemset((i,j),i)
        mapy.itemset((i,j),j)
rst=cv2.remap(img,mapx,mapy,cv2.INTER_LINEAR)
cv2.imshow("original",img)
cv2.imshow("result",rst)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，出现如图 5-11 所示的运行结果，其中，左图是原始图像，右图是互换结果图像。



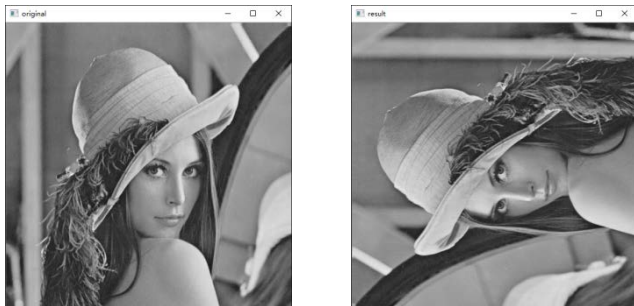


图 5-11 互换结果

5.5.7 图像缩放

上述介绍的映射都是直接完成的整数映射，处理起来比较方便。在处理更复杂的问题时，就需要对行、列值进行比较复杂的运算来实现。

【例 5.20】设计程序，使用函数 `cv2.remap()` 缩小图像。

缩小图像后，可以将图像固定在围绕其中心的某个区域。例如，将其 x 轴、 y 轴设置为：

- 在目标图像的 x 轴($0.25 \cdot x$ 轴长度, $0.75 \cdot x$ 轴长度)区间内生成缩小图像； x 轴其余区域的点取自 x 轴上任意一点的值。
- 在目标图像的 y 轴($0.25 \cdot y$ 轴长度, $0.75 \cdot y$ 轴长度)区间内生成缩小图像； y 轴其余区域的点取自 y 轴上任意一点的值。

为了处理方便，我们让不在上述区域的点都取(0,0)坐标点的值。

根据题目要求，设计程序如下：

```
import cv2
import numpy as np
img=cv2.imread("lena.bmp")
rows,cols=img.shape[:2]
mapx = np.zeros(img.shape[:2],np.float32)
mapy = np.zeros(img.shape[:2],np.float32)
for i in range(rows):
    for j in range(cols):
        if 0.25*cols< i <0.75*cols and 0.25*rows< j <0.75*rows:
            mapx.itemset((i,j),2*( j - cols*0.25 ) + 0.5)
            mapy.itemset((i,j),2*( i - rows*0.25 ) + 0.5)
        else:
            mapx.itemset((i,j),0)
            mapy.itemset((i,j),0)
rst=cv2.remap(img,mapx,mapy,cv2.INTER_LINEAR)
cv2.imshow("original",img)
cv2.imshow("result",rst)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行程序，出现如图 5-12 所示的运行结果，其中，左图是原始图像，右图是缩放结果图像。

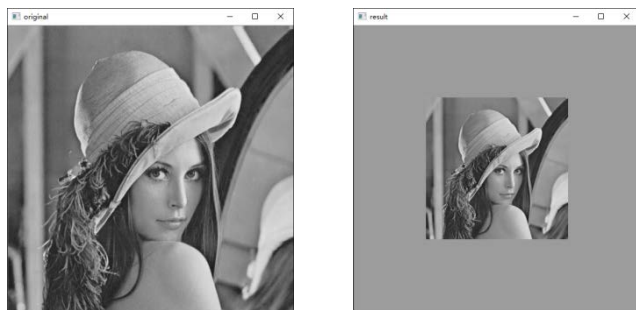


图 5-12 缩放结果